Writing graphs

How the graphs here are put together, and what each craft needs. Start from the nearest existing graph and change it in the editor: that's quicker than writing JSON by hand.

The file

A graph is a PlumCraft workflow file (`docs/interface.md` and `crates/graph` in PlumCraft):

```
{
  "version": 1,
  "engines": ["gridcraft", "cadcraft"],
  "meta": {"title": "Garden planner", "about": "…", "tags": ["garden"]},
  "nodes": {
    "book": {"kind": "gc.new", "pos": [0, 0]},
    "save": {"kind": "file.write", "engine": "gridcraft", "values": {"folder": "out", "basename":
  },
  "wires": [{"from": {"node": "pdf", "port": "pdf"}, "to": {"node": "save", "port": "values"}}]
}
```

Conventions

- **`engines` lists every engine the graph uses.** The headless commands start exactly those engines; the editor needs the same list after `--engine`.
- **`meta.title` and `meta.about` are required.** The about text starts with the engines in the order data flows ("GridCraft CadCraft VectorCraft."), then says what the graph makes and what to change. `docs/catalogue.md` is generated from it.
- **Results go through Write Files** (`file.write`) with `folder: "out"`, a `basename` and `overwrite: true`. The editor writes relative to the graph's folder; `scripts/graphs run` redirects the folder to `out/<graph>/`.
- **Pin each Write Files node to an engine** (`"engine": "gridcraft"`), normally the engine its value comes from. Building blocks exist in every engine, and the pin says which one does the writing.
- **Write Files takes files, not arrays or audio.** Encode first: `array.encode_image`, `audio.to_wav`.
- **Inputs from files** use paths relative to the graph (`../../../assets/pictures/santa-rosa-plums.jpg`), so a graph and its assets move together. Prefer graphs that make their own inputs: noise, demo footage, a new workbook, notes.
- **Give nodes titles** when the kind alone doesn't say what they're for: "Bed table", "Delete the white background".
- **Tidy before you commit:** `scripts/graphs tidy graphs/…/your-graph.json`, then `scripts/graphs check`.

What `check` catches

`plumcraft-editor check` (and so `scripts/graphs check`) reports:

- unknown node types, and values for inputs a node doesn't have;
- wires the type rules refuse;
- wire-only inputs with no wire;
- nodes that no Result or Write Files node depends on. These never run, which almost always means a forgotten wire.

It can't catch a parameter the craft rejects at run time (a swatch name, a command's parameters). Run the graph for that.

Moving data between engines

From	To	How
a craft's export PNG/JPEG (IMAGE)	PhotoCraft, VectorCraft or FigCraft	that craft's Open node, image input
a craft's export PNG/JPEG	the array engine	array.decode_image
array	any image-taking craft	array.encode_image → Open
SVG (CadCraft, FigCraft, VectorCraft, array)	VectorCraft	vc.open, svg input
SVG	array	array.rasterize_svg
WAV (SoundCraft, FilmCraft)	audio	audio.from_wav
a table (audio transients, text letter counts) or an array (histograms)	GridCraft	Table to Rows (seq.table_rows, pinned to the source engine) → gc.write_range values
GridCraft cells	any ~JSON input	gc.read gives [[…], …] rows; gc.evaluate gives one value

Dead ends, as of PlumCraft 0.1.0 (see findings):

- DeckCraft, DesignCraft, WordCraft, EffectCraft, CadCraft, SolidCraft and PdfCraft can't open a picture, though their Open nodes show an image input.
- TXT and MD exports don't convert to TEXT.
- A node's id output (INT) can't wire into another node's ids input (JSON).

Selection

Commands that act on objects take select: all (the default), none, ids:12,14, or, in crafts that can search, color:#ffffff, kind:Path, name:….

- **VectorCraft** supports all of them. vc.run with select: "color:#ffffff" and command: "edit.clear" deletes the white background that CadCraft's SVG export brings along.
- **DesignCraft, FigCraft and CadCraft** only do all and ids. Order the chain so that all means what you want at that step: style the headline while it's the only text, add the shadow while the background is the only shape. Use Frame selection at the end to group what's there.

Per craft

Craft	Notes
CadCraft	Millimetres, y up. Arrays act on select, so draw the thing to repeat first and the rest after it. Exports are fitted to the page; the DWG is true scale.
DeckCraft	dk.new starts with an empty slide: Delete slide 0 after Slides from outline.
DesignCraft	Points, y down, Letter is 612 × 792. Swatch names are bracketed for built-ins ([Paper], [Black]). Text frames use two ids each (the frame and its story).
EffectCraft	#1 is the newest layer. Text and shape positions are centres. Add a solid layer as the background: the composition colour isn't exported. Expressions are JavaScript (time, Math).
FigCraft	props.fills takes ["#RRGGBB"].
FilmCraft	Text position is the left edge. Demo footage gives an item id; wire it to Place on timeline.
GridCraft	chart.set forgets the title unless you pass it. Set a print area when there are hidden columns. Autofit needs cols.
PdfCraft	Opens PDFs only.

Craft	Notes
PhotoCraft	Accepts an `image` input on Open.
SolidCraft	Lengths are strings (expressions): `"16"`, `"L/2"`. `sphere` with `operation: cut` makes pips and dimples; `intersect` rounds corners.
SoundCraft	960 ticks a beat. `drum_synth` uses General MIDI notes (36 kick, 38 snare, 42 hat).
VectorCraft	173 nodes: the whole Effect menu is there. Image Trace presets: Default, 3/6/16 Colors, High/Low Fidelity Photo, Shades of Gray, Black and White Logo.
WordCraft	`wc.new` templates have real placeholder text (Your Name, Recipient Name): fill them with Replace all.
array	HSV hue is in degrees. Variadic inputs (`stack.c`) are wired by slot: `c1`, `c2`, `c3`.